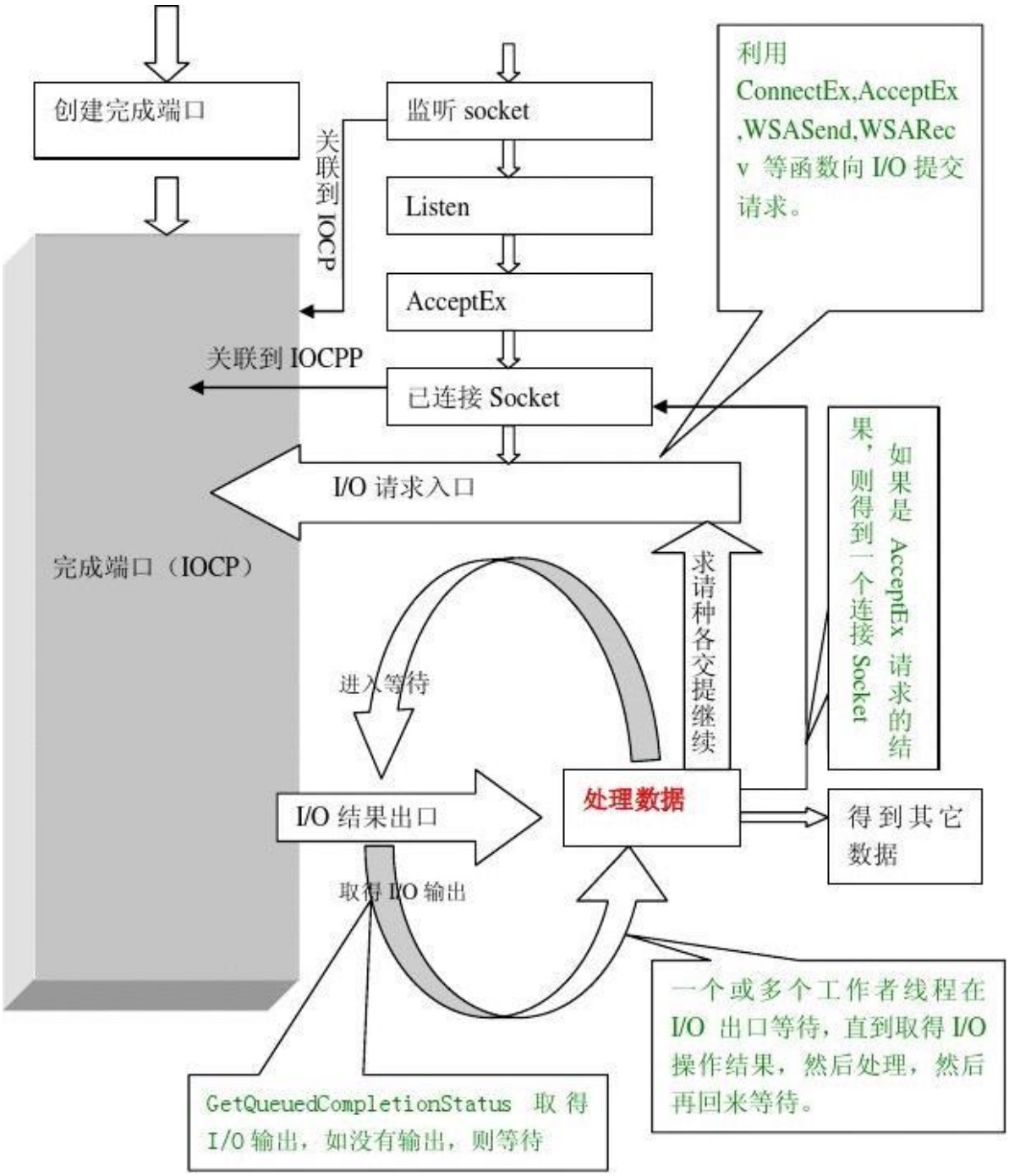
# 零声教育出品 Mark 老师 QQ：2548898954

# 完成端口（IOCP）

> 完成端口不是指物理上的端口，也不是指网络中的端口，而是指操作系统所提供的一种机制。
>
> 这种机制是操作系统提供一种高效处理 io 操作结果的通知机制。总之，完成端口是一种实现高效异步 io 的机制。

## 原理图




# 重叠 io

> 无需等待上一个 IO 操作完成就可以提交下一个 IO 操作请求。也就是这些 IO 操作可以堆叠在一起。
>
> 注意：尽管 IO 操作是按顺序投递的，但是 IO 操作完成通知可以是随机无需的（在多线程等待 IO 完成通知时）。

# 重要接口

- CreateIoCompletionPort

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/ioapiset/nf-ioapiset-createiocompletionport](https://learn.microsoft.com/zh-cn/windows/win32/api/ioapiset/nf-ioapiset-createiocompletionport)
>
> 该接口可以仅创建 IO 完成端口，也可以将现有 IO 完成端口与支持重叠 IO 的任何句柄（TCP 套接字，文件，命名管道等）相关联。

```
// 头文件：ioapiset.h
HANDLE CreateIoCompletionPort(
  [in]            HANDLE    FileHandle,
  [in, optional] HANDLE ExistingCompletionPort,
  [in]            ULONG_PTR CompletionKey,
  [in]            DWORD NumberOfConcurrentThreads
);
```

- AcceptEx

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nf-mswsock-acceptex](https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nf-mswsock-acceptex)
>
> **AcceptEx** 函数接受新连接，返回本地和远程地址，并接收客户端应用程序发送的第一个数据块。

```
BOOL AcceptEx(
  [in]  SOCKET       sListenSocket,
  [in]  SOCKET       sAcceptSocket,
  [in]  PVOID        lpOutputBuffer,
  [in]  DWORD        dwReceiveDataLength,
  [in]  DWORD        dwLocalAddressLength,
  [in]  DWORD        dwRemoteAddressLength,
  [out] LPDWORD      lpdwBytesReceived,
  [in]  LPOVERLAPPED lpOverlapped
);
```

- ConnectEx

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nc-mswsock-lpfn_connectex](https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nc-mswsock-lpfn_connectex)
>
> **ConnectEx** 函数与指定的套接字建立连接，并可以选择在建立连接后发送数据。 仅在面向连接的套接字上支持 **ConnectEx** 函数。

```
BOOL ConnectEx(
  [in]            SOCKET s,
  [in]            const sockaddr *name,
  [in]            int namelen,
  [in, optional] PVOID lpSendBuffer,
  [in]            DWORD dwSendDataLength,
  [out]           LPDWORD lpdwBytesSent,
  [in]            LPOVERLAPPED lpOverlapped
)
```

- DisconnectEx

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nc-mswsock-lpfn_disconnectex](https://learn.microsoft.com/zh-cn/windows/win32/api/mswsock/nc-mswsock-lpfn_disconnectex)
>
> **DisconnectEx** 函数关闭套接字上的连接，并允许重用套接字句柄。

```
BOOL DisconnectEx(
  SOCKET s,
  LPOVERLAPPED lpOverlapped,
  DWORD dwFlags,
  DWORD dwReserved
)
```

- WSARecv

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/winsock2/nf-winsock2-wsarecv](https://learn.microsoft.com/zh-cn/windows/win32/api/winsock2/nf-winsock2-wsarecv)
>
> **WSARecv** 函数从连接的套接字或绑定的无连接套接字接收数据。

```
int WSAAPI WSARecv(
  [in]      SOCKET s,
  [in, out] LPWSABUF lpBuffers,
  [in]      DWORD dwBufferCount,
  [out]     LPDWORD lpNumberOfBytesRecvd,
  [in, out] LPDWORD lpFlags,
  [in]      LPWSAOVERLAPPED lpOverlapped,
  [in]      LPWSAOVERLAPPED_COMPLETION_ROUTINE lpCompletionRoutine
);
```

- WSASend

> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/winsock2/nf-winsock2-wsasend](https://learn.microsoft.com/zh-cn/windows/win32/api/winsock2/nf-winsock2-wsasend)
>
> **WSASend** 函数在连接的套接字上发送数据。

```
int WSAAPI WSASend(
  [in]  SOCKET                                s,
  [in]  LPWSABUF                              lpBuffers,
  [in]  DWORD                                 dwBufferCount,
  [out] LPDWORD                               lpNumberOfBytesSent,
  [in]  DWORD                                 dwFlags,
  [in]  LPWSAOVERLAPPED                       lpOverlapped,
  [in]  LPWSAOVERLAPPED_COMPLETION_ROUTINE    lpCompletionRoutine
);
```

- GetQueuedCompletionStatus

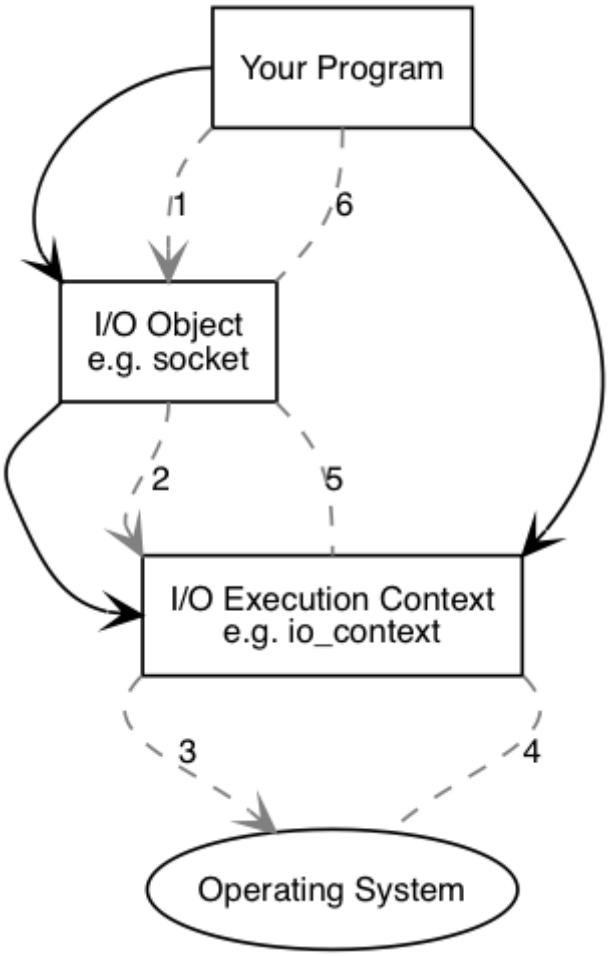
> 具体描述：[https://learn.microsoft.com/zh-cn/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus](https://learn.microsoft.com/zh-cn/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus)
>
> 尝试从指定的 I/O 完成端口取消对 I/O 完成数据包的排队。如果没有完成数据包排队，函数将等待与完成端口关联的挂起 I/O 操作完成。
>
> 此函数将线程与指定的完成端口相关联。 一个线程最多可以与一个完成端口相关联。

```
BOOL GetQueuedCompletionStatus(
  [in]  HANDLE        CompletionPort,
        LPDWORD       lpNumberOfBytesTransferred,
  [out] PULONG_PTR    lpCompletionKey,
  [out] LPOVERLAPPED *lpOverlapped,
  [in]  DWORD         dwMilliseconds
);
```

# boost.asio

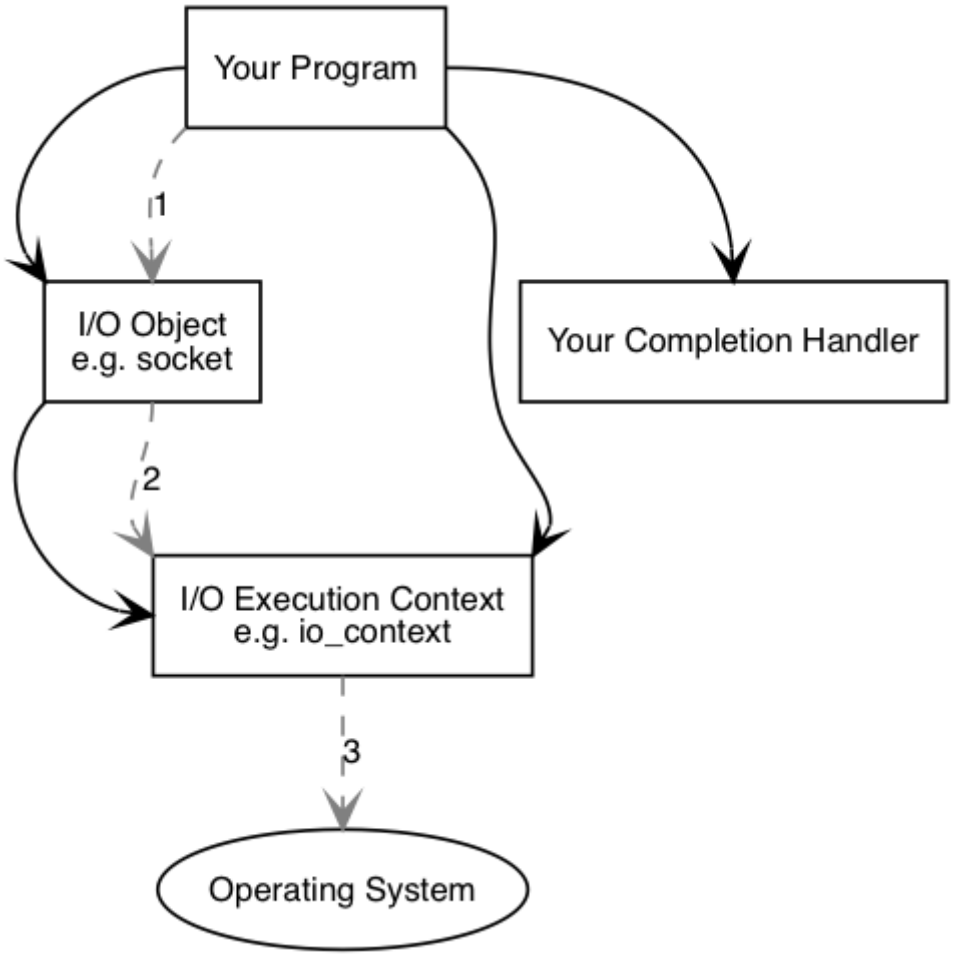
> boost.asio 是一个跨平台的 C++ 网络库。支持同步 IO 和异步 IO。

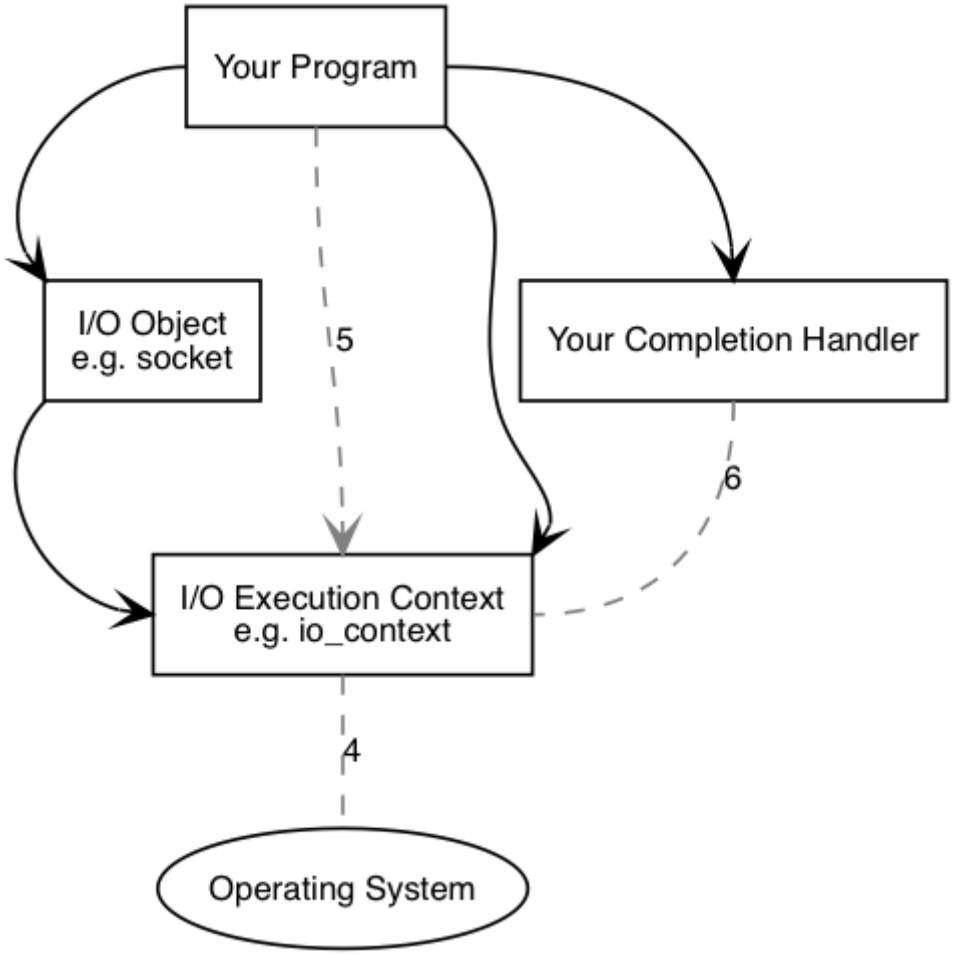
## 同步 IO




```
boost::asio::io_context io_context;
boost::asio::ip::tcp::socket socket(io_context);
socket.connect(server_endpoint); // 将会抛出异常
boost::system::error_code ec;
socket.connect(server_endpoint, ec); // 这里不会抛出异常
```

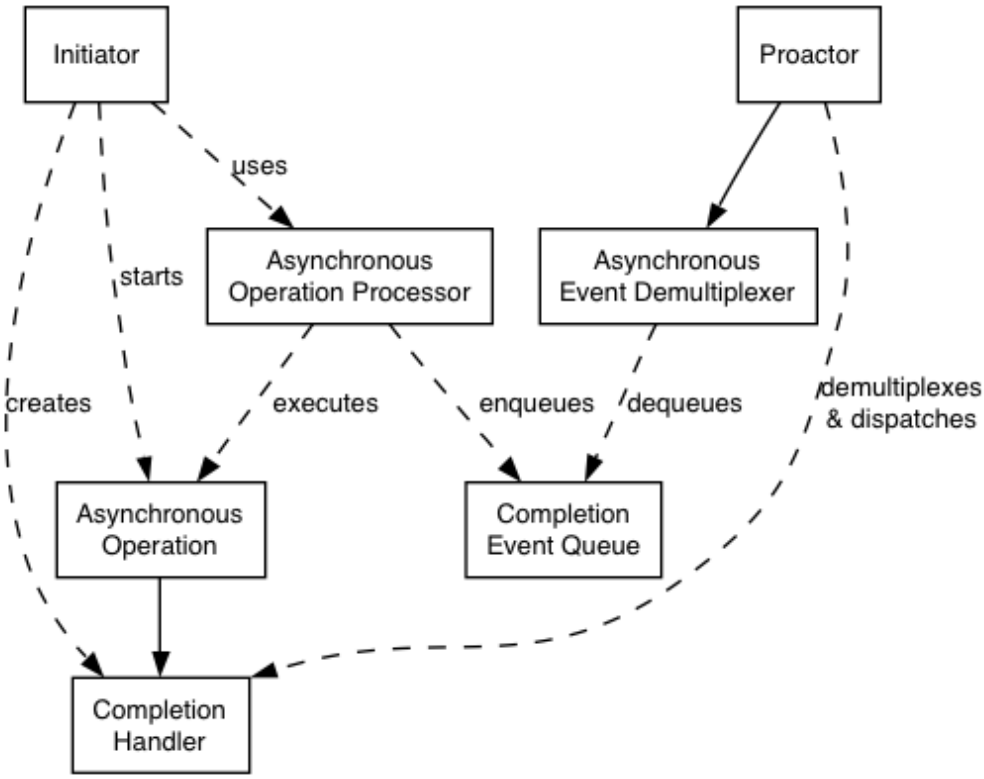
# 异步 IO


Your Program
1
I/O Object
e.g. socket
Your Completion Handler
2
I/O Execution Context
e.g. io_context
3
Operating System


Your Program
I/O Object
e.g. socket
5
Your Completion Handler
6
I/O Execution Context
e.g. io_context
4
Operating System

# proactor 模型

> 详解在：[https://www.boost.org/doc/libs/1_82_0/doc/html/boost_asio/overview/core/async.html](https://www.boost.org/doc/libs/1_82_0/doc/html/boost_asio/overview/core/async.html)。
>
> 其中：
>
> - Asynchronous Operation Processor
>
>  reactor 模型中， 相当于 select、poll、epoll。
>
>  IOCP 中， 相当于 AcceptEx、WSARecv、WSASend等。




# 基本概念及接口

- io_context

  > 类似 reactor 对象，或者 iocp 对象。

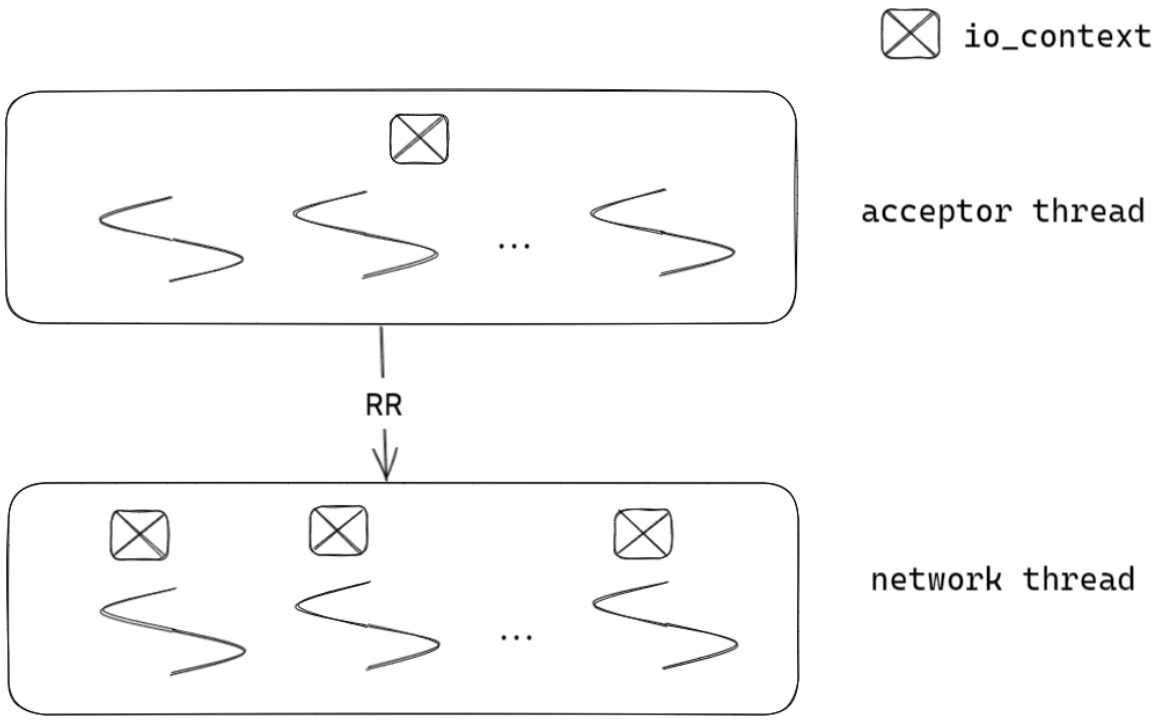
  - run

- ip::tcp::acceptor
  - async_accept

- ip::tcp::endpoint：端点，某个地址和端口的封装。
- ip::tcp::socket：socket创建时，需要与 io_context 进行绑定。
  - async_connect
  - async_read_some
  - async_write_some

# TrinityCore

## 原理图




- AsncAcceptor.h

> 封装 tcp::acceptor，并实现负载均衡机制，将接收的 socket 转发到 network 线程；

- NetworkThread.h

> 封装读写操作处理流程以及管理 socket，每1ms更新一次 socket。

- Socket.h

  > 封装单个连接，定制读写操作，考虑到跨平台处理的差异。

- SocketMgr.h

  > 管理整个网络模块的抽象，提供连接打开后的操作。